

Grado en Ingeniería Informática
Fundamentos de Programación II 25/26

Hito 4:
Recuperación del proyecto práctico

Convocatoria extraordinaria

Corrección de H1–H3 y ampliación final

Resumen

Este hito está destinado al alumnado que debe recuperar el proyecto práctico en convocatoria extraordinaria. La entrega no consiste en realizar una práctica independiente, sino en presentar una versión completa, corregida e integrada del proyecto desarrollado en los hitos 1, 2 y 3, añadiendo además una ampliación final de alcance reducido.

IMPORTANTE: La recuperación debe entregar un proyecto completo y funcional. No basta con implementar únicamente la nueva funcionalidad del Hito 4 si el sistema no conserva correctamente el comportamiento exigido en los hitos anteriores.

1. Objetivos

- Corregir e integrar el trabajo realizado en los hitos 1, 2 y 3 en un único proyecto coherente.
- Mantener la funcionalidad previa del videojuego RPG: personajes, inventarios, tienda, equipamiento, ficheros y excepciones.
- Añadir una nueva ampliación basada en límite de peso y transferencia segura de objetos.
- Garantizar que las operaciones que fallan no dejan el sistema en un estado inconsistente.
- Consolidar el proceso de trabajo por fases: análisis, diseño e implementación.

2. Normas de trabajo

El ejercicio se entregará en la tarea habilitada en Campus Virtual para la convocatoria extraordinaria. Salvo indicación expresa del profesorado, se deberán seguir estas normas:

■ Entrega

- Todos los archivos deberán comprimirse en un único archivo ZIP.
- El nombre del ZIP debe seguir el formato `LX_XX_H4_EXTRA.zip` según el grupo de Campus Virtual.
- El ZIP deberá incluir:
 - Memoria final de recuperación, explicando qué se ha corregido de H1–H3 y qué se ha añadido en H4.
 - Imagen del diagrama de clases y relaciones UML (`.png` o `.jpg`).
 - Únicamente la carpeta `src` con los archivos `.java`.

■ Restricciones de implementación

- No utilizar múltiples `return` en una misma función.
- No utilizar `continue`.

- No utilizar `break`, salvo en sentencias `switch`.
- No utilizar `ArrayList`.
- Queda totalmente **prohibido el uso de Inteligencia Artificial** para realizar estas prácticas. Cualquier indicio de su utilización será penalizado.

3. Fases del desarrollo orientado a objetos

Se pide seguir las fases del desarrollo de software orientado a objetos:

- **Análisis:** En esta fase se identifican los requisitos del sistema. Intentamos encontrar los objetos y relaciones adecuados para el problema propuesto. Investigamos el problema centrándonos en los conceptos del dominio del problema, sin preocuparnos por la implementación. Es importante entender el problema y definir claramente los requisitos antes de pasar a la siguiente fase.
- **Diseño:** En esta fase se define la arquitectura del sistema. Partiendo de los objetos y relaciones identificados en el análisis, se diseña la estructura del sistema. Se definen las clases, sus atributos y métodos (aún sin pensar en la implementación, solo en aquella funcionalidad que cada clase debe tener), así como las relaciones entre ellas (herencia, composición, etc.). Es importante diseñar un sistema modular y flexible que permita cambios futuros sin afectar a toda la aplicación. En esta fase es fundamental el uso de diagramas UML (especialmente el diagrama de clases).
- **Implementación:** Únicamente cuando tenemos claro el diseño, pasamos a la implementación. En esta fase se escribe el código fuente de la aplicación siguiendo el diseño establecido. Es importante seguir buenas prácticas de programación y mantener el código limpio y organizado.

4. Desarrollo de la práctica

La práctica se divide en dos partes. La primera parte consiste en recuperar el proyecto anterior, corrigiendo los problemas de análisis, diseño o implementación que impidan que el sistema funcione como se pedía en H1, H2 y H3. La segunda parte añade una ampliación final que obliga a razonar sobre el estado del sistema cuando un objeto se mueve de un lugar a otro.

4.1. Parte A: recuperación de H1, H2 y H3

El proyecto entregado debe conservar, al menos, la funcionalidad exigida en los hitos anteriores:

- **Hito 1:** personaje, objetos básicos, armas, metales, inventario de tamaño fijo, equipar y desequipar arma, y menú por consola.
- **Hito 2:** varios personajes, tienda, compra de objetos, armaduras, interfaz de equipamiento, `enum` para los tipos de armadura y equipo de cuatro posiciones.
- **Hito 3:** excepciones propias del dominio, gestión de errores de entrada/salida, exportación del personaje actual a fichero y mantenimiento de un formato coherente con los ficheros del sistema.

El código debe compilar, arrancar correctamente y permitir usar el menú principal sin errores iniciales. Si una funcionalidad previa se rompe al añadir H4, se considerará un problema de integración.

4.2. Parte B: ampliación del Hito 4

En este hito se añade un límite de peso al inventario de cada personaje y se incorporan operaciones seguras de movimiento de objetos entre contenedores. La idea general de estas operaciones es:

`origen → objeto → destino`

El origen puede ser el inventario de un personaje. El destino puede ser la tienda o el inventario de otro personaje. La operación debe comprobar todas las condiciones necesarias antes de modificar definitivamente el estado del sistema.

4.2.1. Límite de peso del inventario

Cada personaje tendrá un peso máximo de inventario de **20 unidades**. Para calcular el peso ocupado se sumará el campo `peso` de todos los objetos almacenados en su inventario. No se deben inventar nuevas fórmulas de peso ni recalcular el peso a partir de otros atributos.

Se deberá añadir una nueva excepción propia:

- `PesoMaximoSuperadoException`: se producirá cuando se intente añadir a un inventario un objeto que haría que se superase el peso máximo permitido.

El menú deberá permitir consultar, junto con el inventario del personaje, el peso ocupado y el peso máximo disponible.

4.2.2. Venta de objetos a la tienda

El personaje podrá vender a la tienda un objeto de su inventario. Al realizar la venta:

- el objeto se elimina del inventario del personaje;
- el objeto se añade al inventario de la tienda;
- el personaje recibe como dinero el precio del objeto vendido.

La venta solo debe completarse si la tienda tiene hueco para recibir el objeto. Si la posición elegida está vacía o la tienda no tiene huecos libres, la operación no se realiza y el personaje no recibe dinero.

4.2.3. Transferencia de objetos entre personajes

También se deberá permitir transferir un objeto desde el inventario de un personaje al inventario de otro personaje. En esta versión la transferencia entre personajes será gratuita: no hay intercambio de dinero.

La transferencia solo debe completarse si:

- la posición de origen contiene un objeto;
- el personaje destino tiene hueco libre en el inventario;
- el objeto no hace que el personaje destino supere su peso máximo.

Si alguna de estas condiciones falla, el objeto debe permanecer en el inventario del personaje origen.

4.3. Consistencia de las operaciones

Las nuevas operaciones deben ser seguras. Esto significa que no se puede perder ni duplicar ningún objeto cuando se produce un error. Tampoco se puede modificar el dinero de un personaje si la venta no ha terminado correctamente.

En particular:

- Si el origen está vacío, se utilizará `PosicionVacíaException` o una excepción equivalente ya definida en el proyecto.
- Si el destino no tiene hueco, se utilizará `InventarioLlenoException`.
- Si el destino supera el peso máximo, se utilizará `PesoMaximoSuperadoException`.
- Si una operación falla, el estado del sistema debe quedar igual que antes de intentar la operación.

4.4. Ficheros y restricciones

No se debe cambiar el formato de los ficheros `personajes.txt` y `tienda.txt`. El campo `peso` ya existe en los objetos y debe utilizarse tal como aparece en los ficheros.

La ampliación debe seguir trabajando con arrays de tamaño fijo y posiciones vacías representadas mediante `null`. No se permite usar `ArrayList`, `HashMap` ni otras colecciones dinámicas.

Se pide actualizar el diagrama UML para reflejar los cambios introducidos en esta recuperación, adaptar las clases necesarias e incorporar las nuevas opciones al menú. La memoria final deberá explicar qué problemas se han corregido respecto a los hitos anteriores y cómo se ha diseñado la nueva funcionalidad del Hito 4.